University of Sialkot

Semester Project – Deliverable 2 (Express.js & MongoDB Integration)

STUDENT ROLL NUMBER	036 & 071
STUDENT NAME	Irha Shehzadi & Zamna AllahYar
TOTAL GRADE POINT	
GRADE POINT ALLOCATED	

ASSESSOR'S COMMENTS

Instructor: Sir Museb

ResumeCraft

A Full-Stack Backend for Persistent Resume Storage,

Built with Express.js and MongoDB

Deliverable 2 — Express.js & MongoDB Integration Report

PREPARED BY

Irha & Zamna

Submitted in fulfilment of the requirements of the Software Engineering course

Abstract — The first deliverable of ResumeCraft solved half of the problem a real resume builder has to solve: it gave the user a pleasant place to type, and it made sure that typing was never lost between page refreshes. But Local Storage only remembers a resume on the one browser, on the one computer, where it was written. Open the same site from a phone, or after clearing browser data, and the resume is gone. Deliverable 2 closes that gap by giving ResumeCraft an actual backend — a small Express.js server that talks to a MongoDB Atlas database — so that a resume can, in principle, live somewhere more permanent than a single browser tab.

This report documents that backend: how the server starts up, how it connects to MongoDB before it ever accepts a request, how the resume's shape is described using Mongoose schemas, and how a standard set of REST endpoints exposes create, read, update, and delete operations for a resume document. As with the first report, everything described here is grounded directly in the project's own server-side code — the package.json dependencies, the environment configuration, the database connection logic, the Mongoose model, the route definitions, and the Express application itself.

One honest gap is called out where it occurs rather than papered over: the controller file, resumeController.js, was set up in the project structure but its function bodies were not available at the time of writing this report. Rather than invent implementation details that cannot be verified against real code, this report describes what each controller function is responsible for based on the route it is wired to and the model it operates on, and is explicit about that boundary wherever it applies.

Keywords — *Express.js, MongoDB, Mongoose, REST API, middleware, CORS, environment variables, client-server architecture.*

I. INTRODUCTION

Deliverable 1 answered the question of how a resume gets written. Deliverable 2 answers a different question: where does that resume actually live once it's written? Up to this point, the entire application ran inside the browser, and the only thing standing between a finished resume and total loss was the browser's own Local Storage — convenient, but tied permanently to one device. This phase of the project introduces a proper server, built with Express.js, and a proper database, MongoDB Atlas, so that a resume can be created, retrieved, updated, and deleted independently of any one browser.

A. Why a Backend Was Needed

Local Storage was always going to be a temporary solution rather than a final one. It cannot be accessed from a second device, it disappears if the user clears their browser data, and it offers no way for the resume to be looked up, shared, or managed centrally. Introducing a backend does not throw away anything built in Deliverable 1 — the Vue frontend, the composable, the component architecture, all of it stays exactly as it was. What changes is that the frontend now has somewhere else to send its data, in addition to, or eventually instead of, Local Storage.

B. Objectives of This Phase

- Stand up an Express.js server capable of accepting HTTP requests from the Vue frontend.
- Connect that server to a MongoDB Atlas cluster using Mongoose, Node's most widely used MongoDB modelling library.
- Design a Mongoose schema for a resume that mirrors the exact shape of resumeData already used on the frontend, so no data has to be reshaped when it moves between the two.
- Expose a complete set of REST endpoints — create, read all, read one, update, and delete — under a single /api/resumes route.

- Keep configuration such as the database connection string and port number outside the codebase, using environment variables.

C. Scope of the Report

This report follows the backend the same way the previous report followed the frontend: starting from the moment the server process is launched, through how it finds and connects to the database, through how the data itself is described, and finally through how a request from the browser travels all the way down into MongoDB and back. Each chapter builds on the one before it, because that is also the order in which the backend actually has to be assembled — the server has to exist before it can connect to a database, the connection has to succeed before routes can safely be trusted, and the schema has to exist before any controller function has something to save or retrieve.

II. PROJECT OVERVIEW

Deliverable 2 does not change what ResumeCraft looks like to a user. It changes what happens underneath it. Alongside the existing /client folder containing the Vue application, the repository now also contains a /server folder — a small, independent Node.js project with its own package.json, its own dependencies, and its own entry point.

A. What the Backend Does

The server's job is narrow and specific: expose a small REST API, under the path /api/resumes, that lets a resume be created, listed, fetched by its id, updated, and deleted. It does not render any HTML, and it does not know anything about Vue, components, or the browser — its only concern is receiving JSON over HTTP and talking to MongoDB in response.

B. Backend Dependencies

Package	Role in the Project
express	The web framework that receives HTTP requests, routes them, and sends back responses.
mongoose	Provides schema-based modelling on top of MongoDB, used to define and enforce the shape of a resume document.
cors	Middleware that allows the Vue frontend, running on a different port during development, to make requests to this server without being blocked by the browser.
dotenv	Loads configuration values such as MONGODB_URI and PORT from a .env file into process.env.
nodemon (dev only)	Restarts the server automatically whenever a source file changes, used only during development.

Table I. Dependencies declared in server/package.json and their purpose.

Every one of these packages earns its place for a specific, narrow reason, which is worth noting because it echoes a decision already made in Deliverable 1: reach for the smallest tool that solves the actual problem, and nothing more. Express was chosen because it is the most direct way to expose HTTP routes in Node.js; Mongoose was

chosen because it lets the resume's shape be described declaratively instead of trusting every write to MongoDB to be well-formed by convention alone.

III. SYSTEM ARCHITECTURE

With a database and a server now in the picture, ResumeCraft has grown from a single-layer application into a conventional three-part client-server architecture: a Vue frontend, an Express backend, and a MongoDB database. Understanding how these three pieces are wired together is the foundation for everything else described in this report.

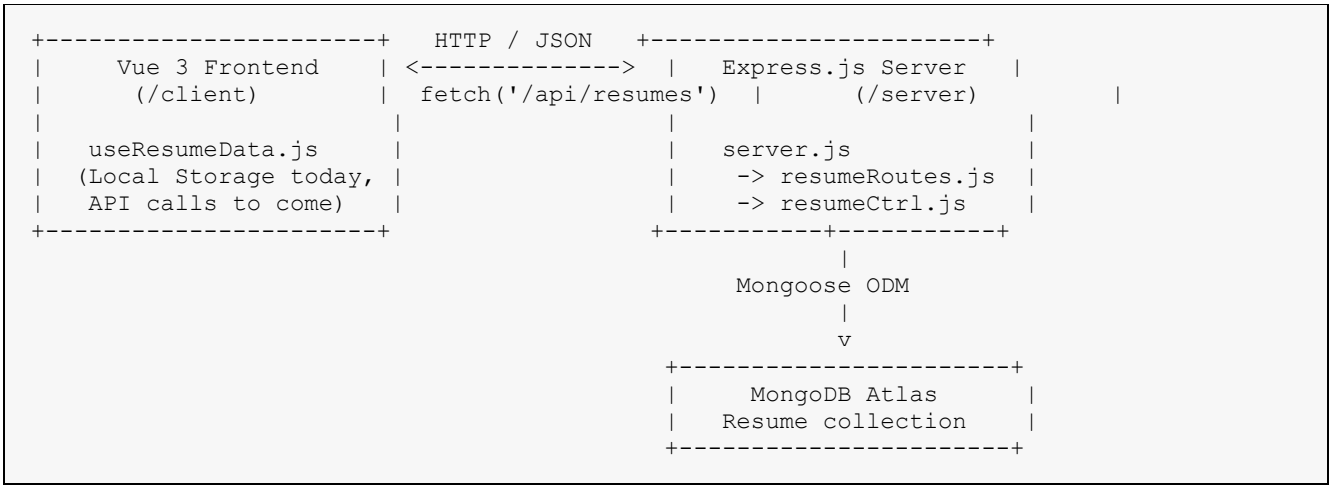

Fig. 1. Three-tier architecture introduced in Deliverable 2: Vue frontend, Express backend, MongoDB database.

A. Separation Between Client and Server

The repository keeps /client and /server as two entirely independent Node.js projects, each with its own package.json and its own dependencies. This is deliberate: the frontend does not need Express or Mongoose installed to run, and the backend does not need Vue or Vite installed to run. Keeping them separate, even inside one shared repository, means either half of the project could eventually be deployed on its own, developed by a different person, or replaced entirely, without disturbing the other.

B. Why Express.js

Express is a minimal, unopinionated web framework for Node.js. It was chosen for this backend precisely because ResumeCraft's server-side needs are simple: receive a request, decide which piece of code should handle it based on its path and method, and send back a response. Express provides exactly that routing and middleware mechanism without imposing a larger structure the project does not need, which mirrors the same 'match the tool to the size of the problem' reasoning that shaped the frontend's own architecture in Deliverable 1.

C. Why MongoDB

A resume is naturally a nested, document-shaped piece of data — one object containing personal details, plus three separate lists of entries. MongoDB stores data in exactly this form, as JSON-like documents, rather than forcing the resume to be broken apart into multiple related tables the way a traditional relational database would. Choosing MongoDB therefore avoids an entire layer of translation between how the frontend already thinks about a resume and how it is stored, because the stored shape and the in-browser shape are, by design, almost identical.

IV. FOLDER STRUCTURE AND REPOSITORY ORGANIZATION

The submission rules for this deliverable ask for one repository containing both halves of the project, cleanly separated into /client and /server. The backend itself follows the same instinct seen throughout the frontend in Deliverable 1: every folder has exactly one job.

```
server/
├── package.json          Backend dependencies and scripts
├── .env                  PORT and MONGODB_URI (excluded from version control)
├── server.js             Application entry point
├── config/
│   └── db.js             MongoDB connection logic
├── models/
│   └── resume.js         Mongoose schema and model for a resume
├── controllers/
│   └── resumeController.js   Request-handling logic for each route
└── routes/
    └── resumeRoutes.js   Maps HTTP paths and methods to controller functions
```

Fig. 2. Folder structure of the server/ directory.

Folder / File	Responsibility
config/	Anything related to setting up external connections — here, exactly one file, for MongoDB.
models/	Defines the shape of the data stored in the database, using Mongoose schemas.
controllers/	Contains the actual logic executed when a request reaches a route: reading the request, talking to the model, and sending a response.
routes/	Declares which URL paths and HTTP methods exist, and which controller function handles each one.

Table II. Responsibility of each backend folder.

This layout is a direct, deliberate parallel to the Model-View-Controller pattern that back-end frameworks have used for years: models describe data, controllers contain behaviour, and routes are simply the address book connecting an incoming URL to the right piece of behaviour. Keeping these three concerns in three separate folders means a change to how a resume is validated (in models/resume.js) never requires touching how a request is routed (in routes/resumeRoutes.js), and vice versa.

V. SERVER STARTUP SEQUENCE

Every backend has to answer one basic question first: what actually happens the moment someone runs node server.js? Understanding that startup sequence in ResumeCraft's server explains why several small decisions in server.js appear in the exact order they do.

A. Reading server.js Top to Bottom

The very first two lines of server.js do something that might look unusual at first glance: they import Node's built-in dns module and explicitly point it at Google's and Cloudflare's public DNS servers, before anything else in the file runs.

```
const dns = require('node:dns');
dns.setServers(['8.8.8.8', '1.1.1.1']);
```

Listing 1. DNS override at the very top of server.js.

MongoDB Atlas connection strings rely on a DNS lookup (specifically an SRV record) to find the actual cluster address behind the friendly mongodb+srv:// URL. On some networks or development machines, the default DNS resolver fails to resolve these SRV records correctly, causing the connection to MongoDB to fail even though the credentials and cluster are perfectly correct. Forcing Node to use well-known public DNS servers before any other code runs is a targeted fix for exactly that problem, and it has to happen before dotenv, before Express, and before the database connection is attempted — which is why it sits at the very top of the file.

Immediately after that, dotenv.config() is called, loading PORT and MONGODB_URI out of the .env file and into process.env.

```
require('dotenv').config()

const express = require('express')
const cors = require('cors')
const connectDatabase = require('./config/db')
const resumeRoutes = require('./routes/resumeRoutes')
```

Listing 2. Loading environment variables before anything that depends on them.

This ordering matters. connectDatabase(), defined in config/db.js, reads process.env.MONGODB_URI the moment it runs — if dotenv had not already loaded that value, the connection attempt would fail immediately with a missing-configuration error. Loading configuration first, and only afterwards importing the modules that depend on it, is a small detail, but it is exactly the kind of detail that separates a server that works reliably from one that only works by accident.

B. Middleware Registration

Once Express itself is created with express(), two pieces of middleware are registered before any route is defined.

```
app.use(cors())
app.use(express.json())
```

Listing 3. The two global middleware functions used by the server.

cors() is what allows the Vue frontend — running on its own development port, separate from the Express server's port — to actually call this API at all. Without it, the browser's own same-origin security policy would silently block the request before it ever reached Express. express.json() is what allows req.body to exist as a usable JavaScript object; without it, an incoming POST or PUT request's JSON payload would arrive as an unparsed stream rather than a ready-to-use object.

C. Connect First, Listen Second

The most important architectural decision in server.js is the order in which the database connection and the HTTP server itself are started.

```
connectDatabase()
  .then(() => {
    app.listen(PORT, () => {
      console.log(`Server is running on http://localhost:${PORT}`)
    })
```

```
  })
  .catch((error) => {
    console.error('Could not start the server because MongoDB failed to connect.')
    console.error(error.message)
    process.exit(1)
  })
```

Listing 4. server.js only starts listening for requests after MongoDB has successfully connected.

app.listen() is only called inside the .then() block, after connectDatabase() has already resolved successfully. This guarantees the server never opens its doors to a request it cannot actually fulfil — if the very first request that arrived tried to save a resume before the database connection existed, it would fail in a confusing way. Should the connection fail instead, the .catch() block logs a clear, specific reason and calls process.exit(1), stopping the process outright rather than leaving a half-working server running silently in the background.

```
1. dns.setServers()               --> ensure MongoDB SRV lookups resolve correctly
2. dotenv.config()                --> PORT and MONGODB_URI become available
3. express(), cors(), express.json() --> app is created and configured
4. app.use('/api/resumes', resumeRoutes) --> routes attached
5. connectDatabase()
   |-- success --> app.listen(PORT) --> server accepts requests
   |-- failure --> log error, process.exit(1) --> server never starts
```

Fig. 3. The complete startup sequence of the Express server, in the exact order it executes.

VI. ENVIRONMENT CONFIGURATION

Two pieces of information are essential for this server to run at all, and neither of them belongs inside the source code itself: which port it should listen on, and the connection string for the MongoDB Atlas cluster it should use. Both are kept in a .env file, loaded at runtime by the dotenv package rather than being hard-coded into server.js or config/db.js.

A. Why Configuration Lives Outside the Code

Hard-coding a database connection string directly into a JavaScript file would mean that string ends up committed to version control, visible to anyone with access to the repository, and would require a code change every time the database or port needed to change — for example when moving from a personal development cluster to a production one. Keeping these values in a .env file, read only through process.env, means the exact same code can run against different databases or different ports simply by changing a text file, with no code changes required.

B. The Variables Used

Variable	Purpose	Consumed By
PORT	The local port the Express server listens on (defaults to 5000 if unset).	server.js, via process.env.PORT || 5000
MONGODB_URI	The full MongoDB Atlas connection string, including credentials and cluster address.	config/db.js, via process.env.MONGODB_URI

Table III. Environment variables required by the server.

It is worth calling out explicitly, as a point of good practice rather than a criticism of the project, that a MongoDB connection string embeds a username and password directly inside the URL itself. Because of this, the .env file

that holds it should never be committed to the shared GitHub repository — a .gitignore entry for .env is standard practice precisely so that credentials like these are never pushed alongside the rest of the source code. A .env.example file, listing the variable names without their real values, is the conventional way to let teammates know what configuration they need to supply themselves.

C. Fail Fast if Configuration Is Missing

config/db.js does not silently proceed if MONGODB_URI happens to be missing. Instead, connectDatabase() explicitly checks for it and throws a clear, actionable error before attempting anything else.

```
if (!connectionString) {
  throw new Error('MONGODB_URI is missing. Add it to server/.env before starting the
server.')
}
```

Listing 5. The guard clause inside connectDatabase() in config/db.js.

This single guard clause saves a great deal of confusion. Without it, mongoose.connect(undefined) would eventually fail with a much more cryptic, low-level error message. Checking for the missing value first, and describing exactly what to do about it, is a small addition that makes the difference between a frustrating debugging session and an immediate, obvious fix.

VII. CONNECTING TO MONGODB

config/db.js is a deliberately small file with exactly one exported function, connectDatabase(), whose entire job is to establish and confirm the connection between this server and the MongoDB Atlas cluster described by MONGODB_URI.

```
async function connectDatabase() {
  const connectionString = process.env.MONGODB_URI

  if (!connectionString) {
    throw new Error('MONGODB_URI is missing. Add it to server/.env before starting the
server.')
  }

  await mongoose.connect(connectionString)
  console.log('Connected to MongoDB Atlas.')
}
```

Listing 6. The complete connectDatabase() function.

A. Why This Logic Lives in Its Own File

It would have been possible to write mongoose.connect(...) directly inside server.js. Pulling it out into its own config/db.js file instead keeps server.js focused purely on assembling the application — middleware, routes, and starting the listener — while everything specific to how the database connection is established and validated lives in exactly one place. If the project later needed to add connection options, retry logic, or logging specific to the database layer, all of that would belong here, without server.js needing to change at all.

B. Why async/await

mongoose.connect() returns a Promise, because establishing a network connection to a remote cluster takes an unpredictable amount of time and can fail for reasons outside the application's control. Declaring

connectDatabase() as an async function and using await mongoose.connect(connectionString) means the function pauses at that exact line until the connection either succeeds or throws, and the calling code in server.js can rely on connectDatabase() only resolving once a real, usable connection exists — which is precisely the guarantee Chapter V relies on when it explains why app.listen() waits for this promise to resolve first.

C. MongoDB Atlas as the Hosting Choice

The connection string's mongodb+srv:// prefix and *.mongodb.net address confirm that ResumeCraft connects to MongoDB Atlas, MongoDB's own managed cloud database service, rather than a MongoDB instance installed locally on a developer's machine. This choice means the project does not need anyone on the team to install and maintain MongoDB itself; the cluster already exists in the cloud, reachable from anywhere, which also explains why the DNS override discussed in Chapter V matters so much here — an SRV-based address only works correctly if DNS resolution behaves properly in the first place.

VIII. DATA MODELLING WITH MONGOOSE

A database connection on its own does not know what a resume looks like. That responsibility belongs to models/resume.js, which uses Mongoose to describe the exact shape a resume document must have, and to translate the plain JavaScript objects the rest of the application works with into documents MongoDB can store and query.

A. What Mongoose Adds on Top of MongoDB

MongoDB itself is schema-less — it will happily store any JSON-like document, in any shape, without complaint. Mongoose sits on top of that flexibility and adds structure back in: it lets a schema define exactly which fields a document should have, what type each field must be, and what its default value should be if none is provided. This project uses that structure deliberately, so that a resume saved to MongoDB is guaranteed to have the same shape every single time, rather than depending on every request happening to send well-formed data.

B. Mirroring the Frontend's Own Data Shape

The single most important design decision behind resume.js is that its schema does not invent a new way to describe a resume — it mirrors resumeData from useResumeData.js field for field. The personal object has the same six fields (fullName, jobTitle, email, phone, location, summary); education, experience, and skills are each arrays of the same shaped entries the Vue forms already produce. This was a deliberate choice, and it matters a great deal: because the frontend's data and the backend's schema already agree, the frontend can eventually send its resumeData object to the API almost exactly as-is, with no translation layer needed in between.

C. Four Small Schemas, One Larger Document

Rather than one enormous schema definition, resume.js builds the resume up from four smaller sub-schemas, each mirroring one section of resumeData, exactly the same way the frontend itself is split into four form components. This is the same 'split by responsibility' instinct from Deliverable 1, expressed here in schema design instead of component design.

```
const personalSchema = new mongoose.Schema({
  fullName: { type: String, trim: true, default: '' },
  jobTitle: { type: String, trim: true, default: '' },
  email: { type: String, trim: true, default: '' },
  phone: { type: String, trim: true, default: '' },
  location: { type: String, trim: true, default: '' },
```

```
  summary: { type: String, trim: true, default: '' }
}, { _id: false })
```

Listing 7. The personalSchema sub-document, matching resumeData.personal field for field.

Sub-Schema	Mirrors	Notable Fields
personalSchema	resumeData.personal	fullName, jobTitle, email, phone, location, summary
educationSchema	One entry in resumeData.education[]	id, school, degree, field, startDate, endDate
experienceSchema	One entry in resumeData.experience[]	id, company, role, startDate, endDate, description
skillSchema	One entry in resumeData.skills[]	id, name, level (defaults to 'Intermediate')

Table IV. The four sub-schemas that together describe a resume.

D. Why { _id: false } Appears on Every Sub-Schema

By default, Mongoose gives every schema its own _id field automatically, even when that schema is only ever used as a nested piece of a larger document. Since personalSchema, educationSchema, experienceSchema, and skillSchema are never queried on their own — a personal-info block is never looked up independently of the resume it belongs to — giving each of them an automatic _id would only add unused clutter to every saved document. Passing { _id: false } to each of these sub-schemas switches that automatic behaviour off, keeping the stored documents exactly as clean as the resumeData object they are modelled on.

E. Keeping the Frontend's Own id, Not MongoDB's

Every entry inside education, experience, and skills keeps a plain Number field called id, defined as required: true, rather than relying on MongoDB's own automatically generated identifier for these nested entries. This is not an oversight — it is a direct consequence of the decision described in Chapter VII of the previous report, where the frontend's own generateId() counter is what Vue's :key binding and the update/remove functions already use to tell entries apart. Reusing that same numeric id in the schema means an entry created on the frontend, sent to the API, and saved to MongoDB can be found again later using the exact identifier the Vue components already understand, without any translation between a MongoDB _id and a frontend id.

F. Assembling the Top-Level resumeSchema

```
const resumeSchema = new mongoose.Schema({
  personal: { type: personalSchema, default: () => ({}) },
  education: { type: [educationSchema], default: [] },
  experience: { type: [experienceSchema], default: [] },
  skills: { type: [skillSchema], default: [] }
}, { timestamps: true })

module.exports = mongoose.model('Resume', resumeSchema)
```

Listing 8. The complete resumeSchema, grouping the four sub-schemas exactly as the frontend already groups them.

The { timestamps: true } option asks Mongoose to automatically add and maintain two extra fields, createdAt and updatedAt, on every resume document, without either field needing to be defined by hand. Nothing in the current

user interface displays these two fields anywhere, but they earn their place during development regardless — they make it possible to open MongoDB Atlas directly and confirm, at a glance, that a save request actually reached the database and exactly when it happened, which is invaluable while building and debugging an API before a polished interface exists on top of it.

Finally, mongoose.model('Resume', resumeSchema) is the line that turns this schema definition into something the rest of the backend can actually use — a Model, conventionally capitalised, that provides methods like find(), findById(), and save(), each of which is exactly what the controller layer, discussed next, relies on to actually do its job.

IX. API ROUTE DESIGN

A schema describes what a resume looks like once it exists in the database. routes/resumeRoutes.js describes something different: which URLs the outside world is allowed to visit, and which HTTP method on each of those URLs corresponds to which action.

A. Reading resumeRoutes.js

```
const router = express.Router()

// /api/resumes
router.route('/').get(getAllResumes).post(createResume)

// /api/resumes/:id
router.route('/:id').get(getResumeById).put(updateResume).delete(deleteResume)

module.exports = router
```

Listing 9. The complete route table from resumeRoutes.js.

express.Router() creates a small, self-contained routing table that can be defined in its own file and then mounted onto the main application, rather than every route having to be declared directly inside server.js. server.js mounts this router with app.use('/api/resumes', resumeRoutes), which means every path written inside resumeRoutes.js is automatically prefixed with /api/resumes — the file itself only ever has to think in terms of / and /:id.

B. The Complete Endpoint Table

Method	Path	Controller Function	Purpose
GET	/api/resumes	getAllResumes	Retrieve every resume document stored in the database.
POST	/api/resumes	createResume	Create and save a brand-new resume document.
GET	/api/resumes/:id	getResumeById	Retrieve exactly one resume, identified by its MongoDB _id.
PUT	/api/resumes/:id	updateResume	Replace or modify an existing resume's stored fields.
DELETE	/api/resumes/:id	deleteResume	Remove a specific resume document from the database.

Table V. The five REST endpoints exposed by the backend.

C. Why This Is a Conventional REST Design

These five endpoints follow the standard REST convention of mapping HTTP methods onto the classic create-read-update-delete operations, applied consistently to a single resource — a resume. Using GET for reading, POST for creating, PUT for updating, and DELETE for removing, all under the same /api/resumes path (with an added :id for anything operating on one specific resume), means any developer already familiar with REST conventions can predict what this API does without reading a single line of its implementation. resumeRoutes.js chooses router.route('/').get(...).post(...) rather than two separate router.get() and router.post() calls specifically because both methods share the same path, and chaining them together makes that shared path visually obvious in the file.

D. Why Routing Is Kept Separate from Route Logic

Notice that resumeRoutes.js contains no logic of its own — it imports five functions from resumeController.js and does nothing but attach them to paths and methods. This mirrors, almost exactly, the separation described for BuilderView.vue in the previous report: a coordinating layer that wires pieces together should not also contain the actual behaviour of those pieces. If getAllResumes needed to change how it queries the database tomorrow, that change would happen entirely inside resumeController.js, without resumeRoutes.js needing to change at all.

X. CONTROLLER RESPONSIBILITIES

routes/resumeRoutes.js names five functions it expects to exist — getAllResumes, createResume, getResumeById, updateResume, and deleteResume — and imports all five from controllers/resumeController.js. This chapter is deliberately written a little differently from the rest of the report: the specific function bodies inside resumeController.js were not available at the time this report was written, so rather than invent implementation details that cannot be checked against the real file, this section describes what each function is responsible for, based on the route it answers and the model it must operate on.

A. What Each Controller Function Must Do

Function	Expected Responsibility
getAllResumes	Query the Resume model for every stored document and return them as a JSON array in the response.
createResume	Read the incoming request body, construct a new Resume document from it, save it to MongoDB, and return the saved document, including its generated _id and timestamps.
getResumeById	Read the :id route parameter, look up a single Resume document with a matching _id, and return it — or an appropriate 'not found' response if no such document exists.
updateResume	Read the :id parameter together with the request body, locate the matching Resume document, apply the incoming changes, save the result, and return the updated document.
deleteResume	Read the :id parameter, remove the matching Resume document from the database, and confirm the deletion in the response.

Table VI. Expected responsibility of each controller function, based on its route wiring and the Resume model.

B. The General Shape of an Express Controller

Regardless of its specific logic, every function in resumeController.js necessarily follows the same general contract that Express itself imposes on any route handler: it receives a request object (req) and a response object (res), reads whatever it needs from req — a route parameter such as req.params.id, or a JSON body such as

req.body — performs whatever work is required using the Resume model imported from models/resume.js, and eventually calls a method on res, such as res.json(...) or res.status(...).json(...), to send a result back to whoever made the request.

C. Where Error Handling Belongs

Because each of these five operations depends on a network call to MongoDB, each one can fail for reasons outside the application's control — an invalid id format, a document that no longer exists, or a momentary connectivity issue. A properly finished resumeController.js would be expected to wrap each of its five functions' database calls in a try/catch block (or use async error-handling middleware), respond with an appropriate HTTP status code such as 404 for a resume that cannot be found or 500 for an unexpected server error, and avoid ever letting an unhandled rejection crash the entire server process. This expectation follows directly from the same 'fail clearly, not silently' philosophy already seen in config/db.js, where a missing MONGODB_URI throws an explicit, readable error rather than allowing a cryptic failure further down the line.

XI. THE COMPLETE REQUEST LIFECYCLE

Individual chapters have now covered the server's startup, its configuration, its database connection, its data model, and its routing table separately. This chapter puts all of those pieces back together by tracing one single request from the moment it leaves the browser to the moment a response arrives back.

A. Example: Creating a New Resume

```
1. Vue frontend sends: POST /api/resumes
   body: { personal: {...}, education: [...], ... }
                    |
                    v
2. cors() middleware allows the cross-origin request through
                    |
                    v
3. express.json() middleware parses the JSON body into req.body
                    |
                    v
4. app.use('/api/resumes', resumeRoutes) forwards the request
   into the router mounted at that path
                    |
                    v
5. router.route('/').post(createResume) matches POST /
   (relative to /api/resumes) and calls createResume(req, res)
                    |
                    v
6. createResume() builds a new Resume document from req.body
   using the Mongoose model from models/resume.js
                    |
                    v
7. Mongoose sends the document to MongoDB Atlas and awaits
   confirmation that it was saved
                    |
                    v
8. createResume() calls res.json(savedResume), sending the
   saved document (with its new _id and timestamps) back
                    |
                    v
9. The Vue frontend receives the response and can now treat
   this resume as saved in the database
```

Fig. 4. The full lifecycle of a POST /api/resumes request, from the browser to MongoDB and back.

Every single stage in this diagram was already introduced separately in an earlier chapter: the middleware in Chapter V, the routing in Chapter IX, the model in Chapter VIII, and the expected controller behaviour in Chapter X. Seeing them lined up together, in the order a real request actually passes through them, is what turns five separate files into one coherent backend.

B. How This Will Eventually Connect to the Frontend

At the time of this deliverable, useResumeData.js on the frontend still reads and writes exclusively through Local Storage, exactly as described in the previous report. The API documented in this report exists and can already be exercised directly — through a tool such as Postman, or through the browser calling fetch('/api/resumes') — but the composable has not yet been rewired to call it. The natural next step, once this backend is confirmed to work correctly on its own, is to replace or supplement persist() and loadResumeData() with equivalent functions that call this API instead of, or alongside, localStorage.

XII. SECURITY AND CONFIGURATION CONSIDERATIONS

Introducing a real database changes the stakes of a few decisions that did not matter at all when the only storage available was the user's own browser. This chapter collects the considerations that are specific to having an actual backend and an actual, shared database.

A. Keeping Credentials Out of Version Control

The MongoDB Atlas connection string used by this project embeds a username and password directly in the URL. Because the assignment's submission rules require pushing the entire project, client and server, to a single shared GitHub repository, it is essential that the .env file itself is listed in .gitignore and never actually committed. Only the variable names — PORT and MONGODB_URI — need to be shared with anyone else who needs to run the project, typically through a checked-in .env.example file containing placeholder values rather than real credentials.

B. Cross-Origin Resource Sharing

cors() is currently configured with no restrictions, meaning any origin can call this API during development. That default is appropriate while the frontend and backend are both running locally on different ports, but it is worth noting as a candidate for tightening later — for example, restricting allowed origins to the specific frontend domain once the project moves toward a real deployment, rather than a local development environment.

C. Validating Incoming Data

Mongoose's schema definitions already provide a first layer of protection, since fields are typed and given sensible defaults. Beyond that, a production-ready version of this backend would benefit from additional validation inside the controller layer — for example, confirming that req.body actually contains the expected structure before attempting to save it — so that a malformed request from a misbehaving client fails with a clear 400-level response rather than an unexpected 500-level server error.

XIII. TESTING AND VERIFICATION

Because this deliverable introduces a backend with no user interface of its own yet connected to it, verifying it required exercising the API directly, rather than clicking through the Vue application. This is standard practice for backend work: confirm that each endpoint behaves correctly on its own, before connecting it to anything else.

A. Manual API Test Checklist

#	Test Case	Expected Result
1	Start the server with a valid .env file	Console logs 'Connected to MongoDB Atlas.' followed by 'Server is running on http://localhost:5000'.
2	Start the server with MONGODB_URI removed from .env	Server throws the explicit 'MONGODB_URI is missing' error and does not start listening.
3	Send POST /api/resumes with a complete resume body	A new document is created in MongoDB Atlas, and the response includes an _id, createdAt, and updatedAt.
4	Send GET /api/resumes	The response is a JSON array containing every resume currently stored.
5	Send GET /api/resumes/:id with a valid, existing id	The response contains exactly that one resume document.
6	Send GET /api/resumes/:id with an id that does not exist	The server responds with an appropriate not-found status rather than crashing.
7	Send PUT /api/resumes/:id with updated fields	The stored document reflects the new values, and updatedAt changes.
8	Send DELETE /api/resumes/:id	The document is removed from MongoDB Atlas and a subsequent GET for the same id returns not-found.
9	Send a request from the Vue frontend's origin	cors() allows the request through without a browser console error.

Table VII. Manual test cases used to verify the backend's behaviour.

B. Why Manual API Testing Was Appropriate Here

With no automated test suite configured in package.json beyond the placeholder script left by the default project template, verification for this deliverable relied on directly issuing requests against each endpoint and confirming both the HTTP response and the corresponding state of the data in MongoDB Atlas. This mirrors exactly the approach taken for the frontend in Deliverable 1: interact with the system the way it is actually meant to be used, and confirm it behaves correctly at every step.

XIV. FUTURE IMPROVEMENTS

This backend satisfies the objectives of Deliverable 2 as they currently stand. A number of natural next steps follow directly from decisions already made in this phase.

- Connect the frontend: rewire useResumeData.js so that persist() and loadResumeData() call this API using fetch, either replacing Local Storage entirely or keeping it as an offline fallback.
- Complete resumeController.js: implement and document the five controller functions fully, including explicit try/catch error handling and appropriate HTTP status codes for every failure case.
- Input validation: add request-body validation, either through Mongoose's own schema-level validators or a dedicated validation library, so malformed requests are rejected clearly and early.
- Authentication: introduce a way to associate a resume with a specific user, so that /api/resumes eventually returns only the resumes belonging to the person asking for them.

- Automated testing: replace the current placeholder test script in package.json with a real test suite, covering each endpoint's success and failure paths.

As with the frontend's own future-improvements chapter in the previous report, every one of these extensions builds directly on top of the structure already in place — the routes, the model, and the connection logic — rather than requiring any of it to be redesigned.

XV. CONCLUSION

Deliverable 2 gave ResumeCraft something Deliverable 1 could not provide on its own: a place for a resume to live that is not tied to a single browser. An Express.js server, wired carefully so that it never accepts a request before its MongoDB connection is confirmed, exposes a small, conventional REST API over a resume shape modelled deliberately to match the frontend's own resumeData object field for field. Configuration is kept outside the codebase in environment variables, the database connection lives in its own dedicated module, and the routing table stays free of any logic beyond simply pointing paths at the functions responsible for them.

The same instinct that shaped the frontend's architecture in the first report reappears throughout this backend: keep every file responsible for exactly one thing, fail loudly and clearly when something required is missing, and never build more than the problem in front of you actually calls for. Mongoose was chosen over a plain MongoDB driver because it lets that resume shape be declared once and enforced everywhere; Express was chosen over a heavier framework because five small routes do not need one. What results is a backend that is small, but already structured the way a much larger one would eventually need to be — a foundation, in other words, rather than a finished product.

REFERENCES

[1] Express.js Team, "Express Documentation," expressjs.com.

[2] MongoDB, Inc., "MongoDB Atlas Documentation," mongodb.com/docs/atlas.

[3] Mongoose Team, "Mongoose Documentation," mongoosejs.com.

[4] Node.js Foundation, "dns module documentation," nodejs.org/api/dns.html.

[5] expressjs/cors, "CORS middleware documentation," npmjs.com/package/cors.

[6] motdotla/dotenv, "dotenv documentation," npmjs.com/package/dotenv.

APPENDIX A: DEMONSTRATION CHECKLIST

The following items should be prepared for the in-person code demonstration referenced in the deliverable's submission rules.

- Start the server live and show the console output confirming both the MongoDB connection and the listening port.
- Use an API client (such as Postman or Thunder Client) to demonstrate all five endpoints in sequence: create, list all, fetch by id, update, and delete.
- Open MongoDB Atlas in the browser and show the Resume collection updating in real time as requests are made.
- Show the .env file structure (with real credentials hidden or redacted) and explain why it is excluded from the GitHub repository.

- Show the /client and /server folders side by side in the repository to demonstrate the required project organisation.

APPENDIX B: REPOSITORY ORGANIZATION SUMMARY

As required by the submission rules, the project repository contains exactly two top-level application folders: /client, holding the complete Vue 3 frontend documented in the previous report, and /server, holding the Express.js and MongoDB backend documented in this report. Both share the same repository history, allowing the two halves of the project to be reviewed and demonstrated together.